


**Laboratorium**
**Robotika dan Sistem Cerdas**
**Departemen Teknik Komputer**
***Institut Teknologi Sepuluh Nopember***

# Laporan Akhir
# Praktikum Sistem Mikroprosesor dan Sistem Tertanam

## Pengenalan Mikroprosesor

Anak Agung Ngurah Agung Kresna Ananta - 5024241085

2026

# 1 Langkah-Langkah Percobaan

## 1.1 Persiapan

Sebelum praktikum, VSCode, ekstensi PlatformIO, serta driver ST-Link dan CP2102/CH340 sudah terpasang sesuai dokumen `setup_vscode_platformio.md`. Praktikan membuat satu project PlatformIO bernama `modul1-praktikum` yang dipakai untuk Percobaan 1–5. Di setiap percobaan, yang diganti hanya isi `platformio.ini` dan file program di folder `src/`. Setiap kali `platformio.ini` diubah, praktikan menunggu proses *refresh* PlatformIO di status bar selesai sebelum menekan Build atau Upload. Board baru disambungkan ke komputer saat percobaan yang bersangkutan dimulai.

## 1.2 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Percobaan ini bertujuan memahami board STM32 Blackpill dan melakukan flashing program Blink lewat ST-Link dengan framework Arduino (STM32duino). File `platformio.ini` diisi environment `blackpill_f401cc` dengan `platform = ststm32`, `framework = arduino`, `upload_protocol = stlink`, dan `debug_tool = stlink`. Rangkaiannya hanya koneksi ST-Link ke Blackpill memakai empat kabel jumper female-female: SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3. Pin dicocokkan berdasarkan label karena urutan pin ST-Link bisa berbeda dengan Blackpill. LED yang diamati adalah LED onboard di PC13.

Setelah ST-Link tersambung ke Blackpill dan dicolokkan ke USB komputer, Kode Program 1 ditulis di `src/main.cpp`, lalu di-Build dan di-Upload. Praktikan kemudian mengamati LED PC13, yang seharusnya menyala 0,5 detik dan mati 0,5 detik secara berulang.

**Listing 1:** Blink STM32 Blackpill (framework Arduino)

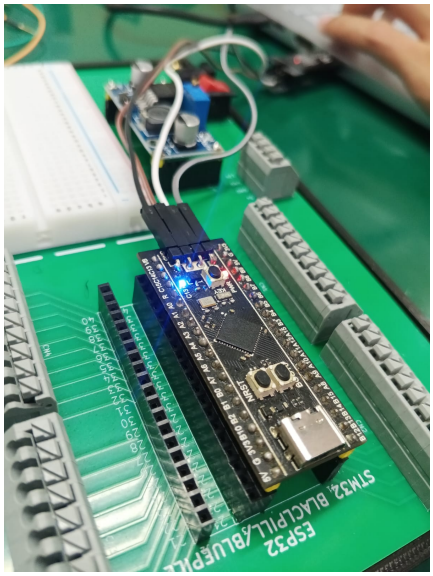
```
#include <Arduino.h>

#define LED_PIN PC13

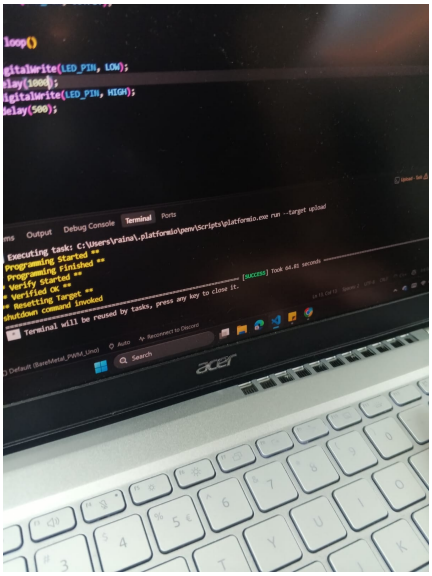
void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
  delay(500);
  digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
  delay(500);
}
```

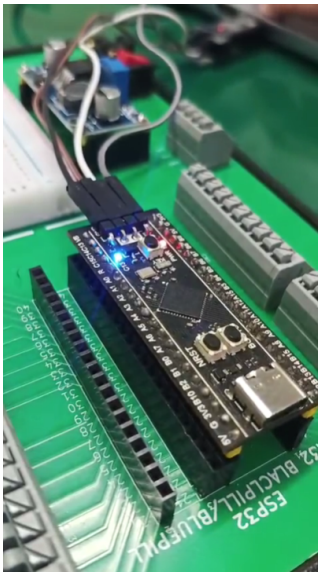
**Tabel 1:** Hasil Percobaan 1



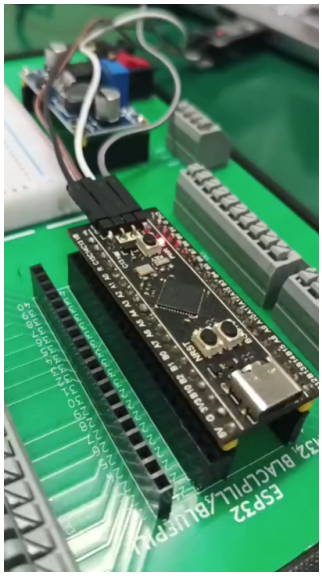
(a) Koneksi ST-Link ke Blackpill



(b) Build dan Upload berhasil



(c) LED PC13 menyala (pin LOW)



(d) LED PC13 mati (pin HIGH)

## 1.3 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Percobaan 2 mengenalkan ESP32 dan struktur program ESP-IDF. File `platformio.ini` diganti ke environment `esp32dev` dengan `platform = espressif32` dan `framework = espidf`. Tidak ada rangkaian tambahan. ESP32 cukup disambungkan ke komputer lewat kabel USB, dan yang diamati adalah LED onboard di GPIO 2.

Setelah port ESP32 terdeteksi, `src/main.cpp` dihapus dan diganti dengan `src/main.c` berisi Kode Program 2. Program di-Build dan di-Upload, lalu praktikan mengamati kedipan LED dan membandingkan cara penulisan program ini dengan program Arduino di Percobaan 1.

**Listing 2:** Blink ESP32 (framework ESP-IDF)

```
1 #include "freertos/FreeRTOS.h"
```

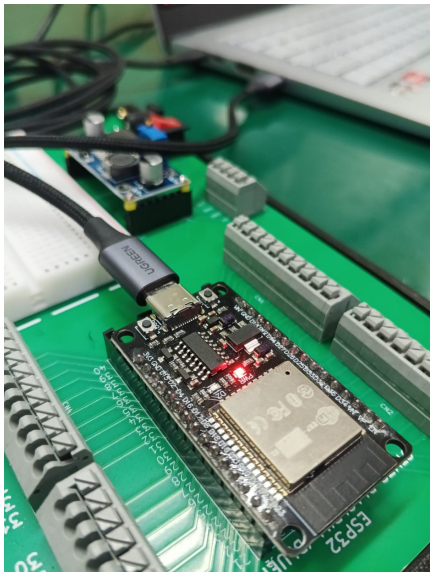
```c
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

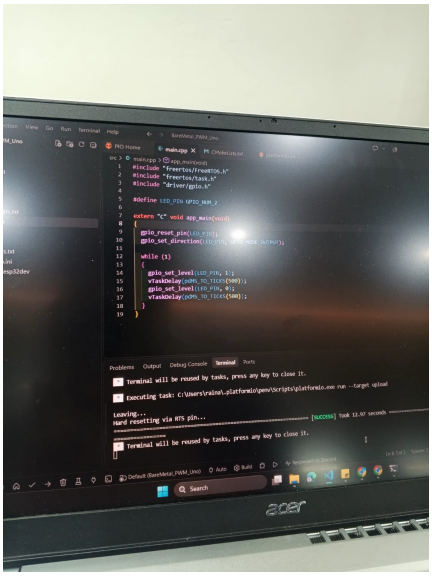
void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

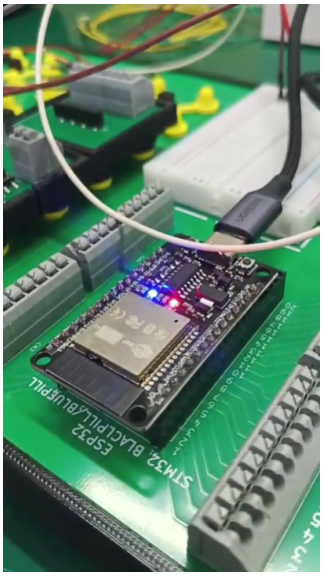
**Tabel 2:** Hasil Percobaan 2



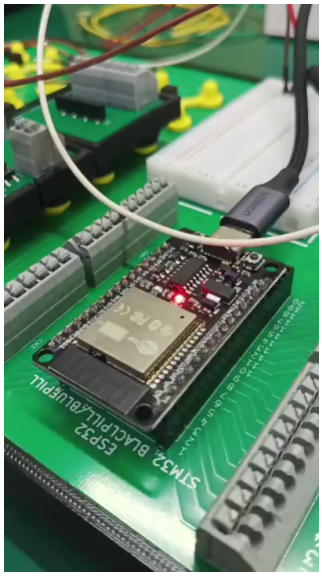
(a) ESP32 tersambung lewat USB



(b) Build dan Upload ESP-IDF berhasil



(c) LED GPIO 2 menyala



(d) LED GPIO 2 mati

## 1.4 Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal (ESP32 + Arduino)

Percobaan 3 membandingkan pembacaan tombol dengan resistor pull-up/pull-down eksternal dan internal pada ESP32. File `platformio.ini` dikembalikan ke `framework = arduino` dengan board `esp32dev`, `src/main.c` dihapus, dan `src/main.cpp` dibuat kembali. Pada Bagian A, pushbutton 1 dipasang di GPIO 32 dengan resistor 10 kΩ ke 3,3 V (pull-up) dan kaki lainnya ke GND. Pushbutton 2 dipasang di GPIO 33 dengan resistor 10 kΩ ke GND (pull-down) dan kaki lainnya ke 3,3 V. Pada Bagian B, kedua resistor dilepas. Tombol di GPIO 32 tetap ke GND, tombol di GPIO 33 tetap ke 3,3 V, dan resistor pull diaktifkan dari program.

Bagian A diuji dengan Kode Program 3. Setelah Upload, Serial Monitor (115200 baud) dibuka dan kedua tombol ditekan bergantian sambil mencatat outputnya. Setelah itu kedua resistor dilepas, pasangan kaki pushbutton tactile diperiksa lagi, dan `setup()` diganti seperti Kode Program 4. Pengujian

yang sama diulang, lalu hasilnya dibandingkan dengan Bagian A. Jika hasilnya berbeda, praktikan memeriksa apakah resistor sudah benar-benar dilepas dan kaki tombol sudah benar.

**Listing 3:** Pull-up dan pull-down eksternal (Bagian A)

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor pull-up dipasang eksternal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor pull-down dipasang eksternal
}

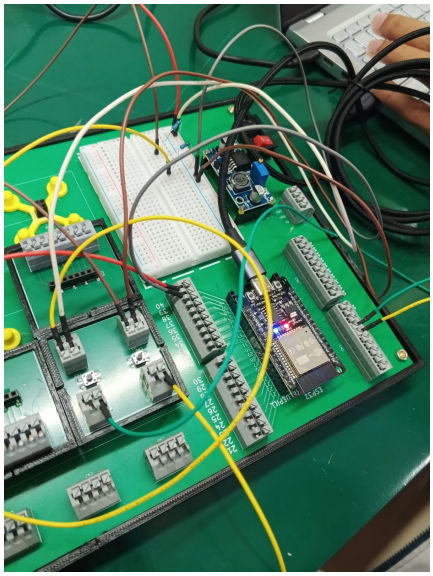
void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

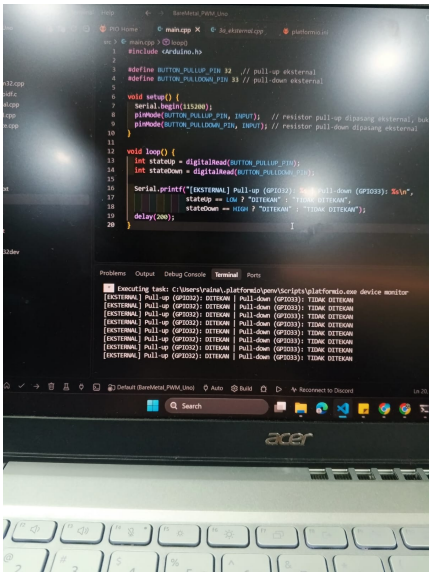
**Listing 4:** Perubahan untuk pull-up dan pull-down internal (Bagian B)

```
void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);     // pull-up internal ESP32
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN); // pull-down internal ESP32
}
// loop() sama dengan Bagian A, label [EKSTERNAL] diganti [INTERNAL]
```

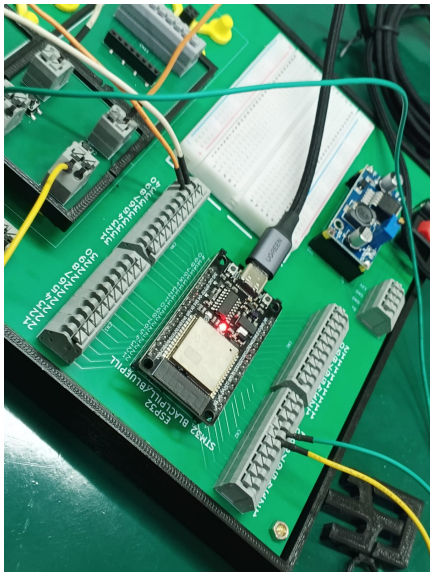
**Tabel 3:** Hasil Percobaan 3



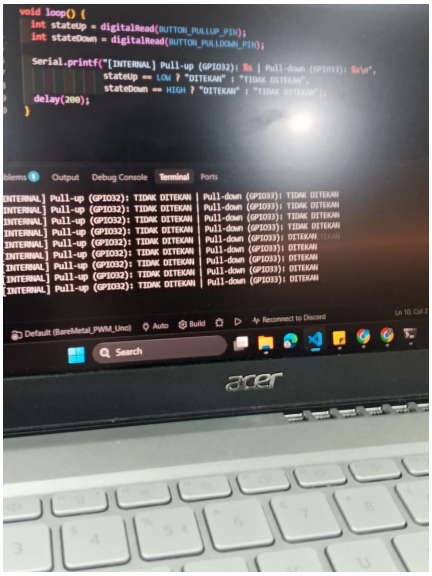
(a) Rangkaian Bagian A (resistor eksternal)



(b) Serial Monitor Bagian A



(c) Rangkaian Bagian B (tanpa resistor)



(d) Serial Monitor Bagian B

## 1.5 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Arduino)

Percobaan 4 menguji efek bouncing pada tombol mekanik dan cara mengatasinya dengan debouncing software. File `platformio.ini` sama dengan Percobaan 3. Satu pushbutton dipasang antara GPIO 25 dan GND dengan pull-up internal (`INPUT_PULLUP`).

Kode Program 5 memakai dua penghitung. `counterNoDebounce` menghitung setiap transisi HIGH ke LOW apa adanya, sedangkan `counterWithDebounce` baru menghitung setelah sinyal stabil selama `DEBOUNCE_DELAY`. Program di-Upload dengan nilai awal 50 ms, lalu tombol ditekan berkali-kali, kadang cepat dan tegas, kadang pelan, sambil membandingkan kedua penghitung di Serial Monitor. Pengujian diulang dengan `DEBOUNCE_DELAY` 10 ms, lalu 200 ms sambil menekan tombol cepat berulang kali. Ketiga hasil kemudian dibandingkan.

**Listing 5:** Perbandingan counter tanpa dan dengan debouncing

```
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms

// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;

// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

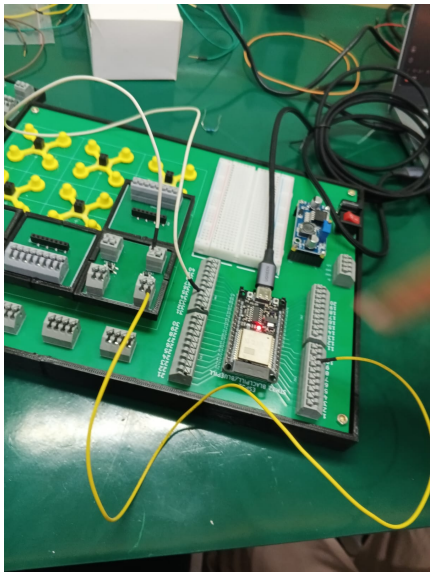
void loop() {
  int reading = digitalRead(BUTTON_PIN);

  // ===== Tanpa Debouncing =====
  if (reading == LOW && rawLastState == HIGH) {
    counterNoDebounce++;
  }
  rawLastState = reading;

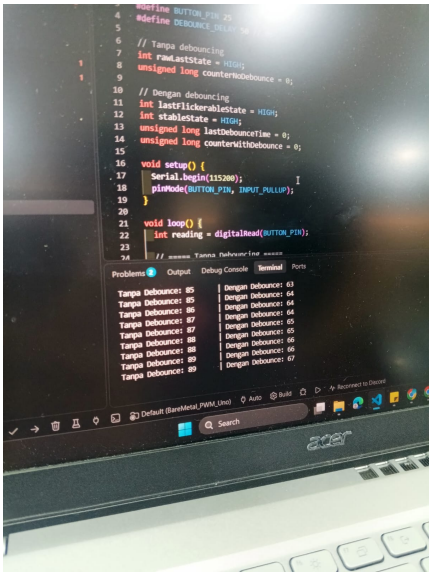
  // ===== Dengan Debouncing =====
  if (reading != lastFlickerableState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (reading != stableState) {
      stableState = reading;
      if (stableState == LOW) {
        counterWithDebounce++;
      }
    }
  }
  lastFlickerableState = reading;

  // ===== Tampilkan kedua nilai secara bersamaan =====
  static unsigned long lastPrintedNoDebounce = 0, lastPrintedWithDebounce = 0;
  if (counterNoDebounce != lastPrintedNoDebounce || counterWithDebounce !=
    lastPrintedWithDebounce) {
    Serial.printf("Tanpa Debounce: %lu\t| Dengan Debounce: %lu\n", counterNoDebounce,
     counterWithDebounce);
    lastPrintedNoDebounce = counterNoDebounce;
    lastPrintedWithDebounce = counterWithDebounce;
  }
}
```

**Tabel 4:** Hasil Percobaan 4



(a) Pushbutton pada GPIO 25



(b) Serial Monitor, delay 50 ms

## 1.6 Percobaan 5: Pembacaan ADC dengan LDR (ESP32 + Arduino)

Percobaan 5 memakai GPIO sebagai input analog melalui ADC dengan sensor LDR. File `platformio.ini` sama dengan Percobaan 3 dan 4. LDR dan resistor 10 kΩ dirangkai sebagai pembagi tegangan: satu kaki LDR ke 3,3 V, kaki lainnya ke GPIO 34 sekaligus ke GND melalui resistor 10 kΩ.

Kode Program 6 membaca nilai ADC mentah (0–4095) dan mengubahnya menjadi tegangan. Setelah Upload, Serial Monitor dibuka dan LDR diuji dalam tiga kondisi: ditutup tangan, cahaya ruangan, dan disinari lampu. Nilai mentah dan tegangan pada tiap kondisi dicatat untuk dianalisis.

**Listing 6:** Pembacaan ADC dengan LDR

```
#include <Arduino.h>

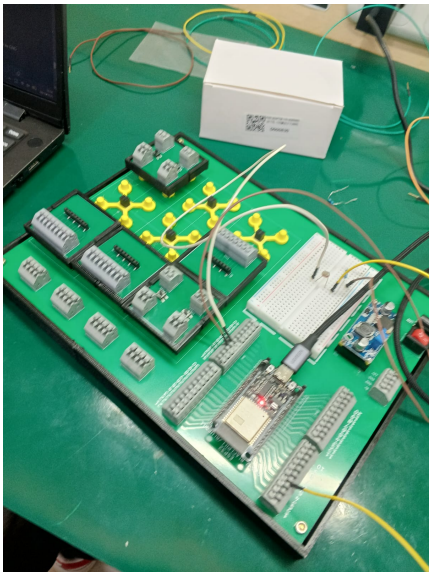
#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

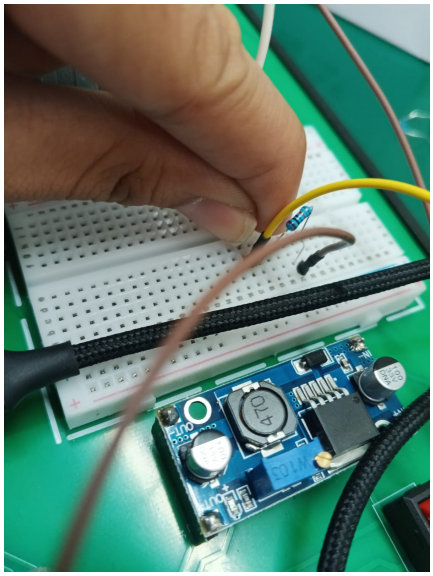
void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

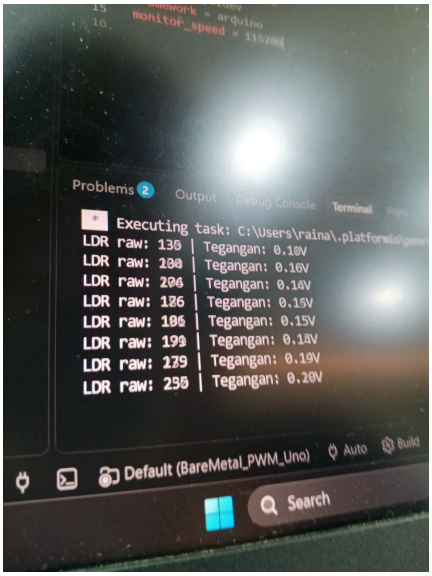
**Tabel 5:** Hasil Percobaan 5



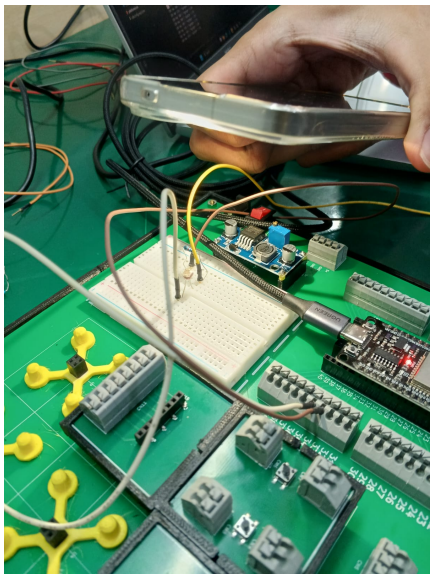
(a) Rangkaian pembagi tegangan LDR



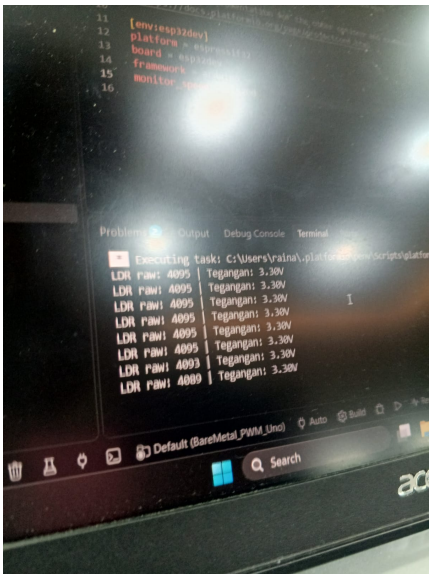
(c) LDR ditutup tangan




(d) Output LDR ditutup tangan



(c) LDR disinari lampu




(d) Output LDR disinari lampu

# 2 Analisis Hasil Percobaan

## 2.1 Percobaan 1: Blink STM32 Blackpill

LED PC13 berkedip dengan periode sekitar 1 detik (Tabel 1), sesuai program. LED onboard Blackpill bersifat aktif LOW: anodanya terhubung ke 3,3 V lewat resistor dan katodanya ke PC13. Saat PC13 bernilai LOW, arus mengalir dan LED menyala. Saat PC13 bernilai HIGH, kedua ujung LED bertegangan sama sehingga LED mati. Karena itu `digitalWrite(PC13, LOW)` di program justru menyalakan LED.

Blackpill tidak punya programmer onboard, jadi flashing harus lewat ST-Link. SWDIO dan SWCLK membawa data dan clock SWD, sedangkan GND menyamakan referensi tegangan antara ST-Link dan board. Pada build pertama, PlatformIO juga harus mengunduh toolchain ARM dan framework STM32duino sebelum program bisa dikompilasi. Upload biasanya gagal jika SWDIO dan SWCLK tertukar, driver ST-Link belum terpasang, atau kabel jumper longgar, sehingga pin dicocokkan berdasarkan label dan bukan berdasarkan posisinya.

## 2.2 Percobaan 2: Blink ESP32 dengan ESP-IDF

LED GPIO 2 berkedip dengan periode 1 detik (Tabel 2). LED onboard ESP32 DevKit aktif HIGH, kebalikan dari Blackpill, sehingga `gpio_set_level(LED_PIN, 1)` menyalakan LED. ESP32 bisa langsung di-flash lewat USB karena board sudah punya chip USB-to-Serial.

Program ESP-IDF tidak punya `setup()` dan `loop()`. Eksekusi dimulai dari `app_main()` yang berjalan sebagai task FreeRTOS, dan perulangan ditulis sendiri dengan `while(1)`. Konfigurasi pin juga lebih eksplisit. Pin di-reset dulu dengan `gpio_reset_pin()`, lalu arahnya diatur dengan gpio_set_direction(), sedangkan di Arduino kedua langkah ini cukup ditulis dengan `pinMode()`. Untuk menulis pin, ESP-IDF memakai `gpio_set_level()` sebagai pengganti `digitalWrite()`.

Delay di ESP-IDF ditulis dengan vTaskDelay(pdMS_TO_TICKS(500)). Fungsi ini menyerahkan CPU ke scheduler FreeRTOS selama delay sehingga task lain tetap bisa berjalan, dan makro `pdMS_TO_TICKS()` mengubah milidetik menjadi jumlah tick. Di Arduino-ESP32, `delay()` sebenarnya juga memanggil `vTaskDelay()`; bedanya, ESP-IDF menampilkan mekanisme FreeRTOS ini langsung di kode.

## 2.3 Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal

Semua pembacaan pada Bagian A dan B sesuai teori (Tabel 3). Tombol di GPIO 32 yang memakai pull-up terbaca HIGH (TIDAK DITEKAN) saat dilepas dan LOW (DITEKAN) saat ditekan. Tombol di GPIO 33 yang memakai pull-down terbaca LOW saat dilepas dan HIGH saat ditekan. Hasil yang sama muncul ketika resistor eksternal 10 k$\Omega$ dilepas dan diganti dengan `INPUT_PULLUP` dan `INPUT_PULLDOWN`.

Pull-up menarik pin ke 3,3 V sehingga pin terbaca HIGH selama tombol tidak ditekan. Saat tombol ditekan, pin terhubung langsung ke GND melalui kontak tombol yang resistansinya jauh lebih kecil dari resistor pull-up, sehingga tegangan pin turun mendekati 0 V dan terbaca LOW. Pull-down bekerja sebaliknya: pin ditarik ke GND dan baru terbaca HIGH ketika tombol menghubungkannya ke 3,3 V. Resistor juga membatasi arus agar VCC dan GND tidak terhubung singkat ketika tombol ditekan.

Kesamaan hasil Bagian A dan B menunjukkan bahwa pull internal ESP32 bisa menggantikan resistor eksternal di GPIO 32 dan 33. Cara ini tidak bisa dipakai di GPIO 34–39 karena pin input-only tersebut tidak punya pull internal. Resistor internal ESP32 bernilai sekitar 45 k$\Omega$ menurut datasheet,

lebih besar dari resistor eksternal 10 k$\Omega$, sehingga tarikannya lebih lemah dan lebih mudah terganggu derau jika kabelnya panjang. Pada percobaan ini kabelnya pendek, jadi perbedaan tersebut tidak terlihat di Serial Monitor.

## 2.4 Percobaan 4: Debouncing

Percobaan dilakukan dengan `DEBOUNCE_DELAY` 50 ms (Tabel 4). Setelah tombol ditekan jumlah kali, `counterNoDebounce` mencapai nilai, sedangkan `counterWithDebounce` mencapai nilai. Kontak logam di dalam tombol memantul beberapa kali selama beberapa milidetik sebelum stabil. Karena `loop()` tidak memakai delay, ESP32 membaca pin sangat sering dan setiap pantulan HIGH ke LOW ikut terhitung oleh `counterNoDebounce`. Pantulan juga terjadi saat tombol dilepas, dan transisi HIGH ke LOW pada fase ini ikut menambah penghitung tanpa debounce.

Penghitung dengan debounce mencatat waktu setiap kali sinyal berubah (`lastDebounceTime`) dan baru memperbarui `stableState` jika sinyal tidak berubah lebih lama dari 50 ms. Selama tombol masih memantul, pewaktu ini terus di-reset, sehingga satu tekanan hanya menghasilkan satu perubahan `stableState` menjadi LOW. Nilai `DEBOUNCE_DELAY` perlu lebih panjang dari durasi pantulan tombol, tetapi cukup pendek supaya respons tombol tidak terasa terlambat. Jika nilainya terlalu kecil, misalnya 10 ms, pantulan yang lebih panjang dari itu masih bisa lolos. Jika terlalu besar, misalnya 200 ms, tombol terasa lambat dan tekanan berulang yang jaraknya kurang dari 200 ms tidak terhitung karena sinyal tidak pernah stabil selama itu. Nilai 50 ms sesuai dengan rentang 20–50 ms yang disarankan pada Dasar Teori C.5.

## 2.5 Percobaan 5: Pembacaan ADC dengan LDR

LDR berada di sisi atas pembagi tegangan dan resistor 10 k$\Omega$ di sisi bawah, sehingga tegangan yang dibaca ADC adalah $V_o = 3{,}3\,\text{V} \cdot R_1/(R_{LDR} + R_1)$ dengan $R_1 = 10\,\text{k}\Omega$. Resistansi LDR turun saat cahaya bertambah, jadi $V_o$ dan nilai ADC seharusnya naik ketika LDR disinari dan turun ketika ditutup.

Hasil pengamatan sesuai dengan pola tersebut (Tabel 5). Saat LDR ditutup tangan, nilai mentah ADC berada di kisaran 130–235 dengan tegangan 0,10–0,20 V. Dari rumus pembagi tegangan, nilai ini setara dengan resistansi LDR sekitar 155–320 k$\Omega$. Nilainya naik turun karena tangan tidak menutup LDR dengan rapat, sehingga sedikit cahaya masih masuk dan jumlahnya berubah setiap kali tangan bergeser. Saat LDR disinari lampu, nilai mentah naik ke 4089–4095 dan tegangan terbaca 3,30 V. Nilai 4095 adalah batas atas ADC 12-bit, jadi ADC sudah jenuh. Jika ADC mulai jenuh di sekitar 3,1 V, resistansi LDR saat itu berada di bawah kira-kira 650 $\Omega$, sehingga hampir seluruh tegangan 3,3 V jatuh di resistor 10 k$\Omega$.

Dua kondisi ini mencakup hampir seluruh rentang ADC, dari sekitar 3% sampai 100% skala penuh. ADC 12-bit membagi 0–3,3 V menjadi 4096 tingkat dengan resolusi sekitar 0,8 mV, sehingga perubahan kecil seperti fluktuasi 130 ke 235 tetap terbaca sebagai nilai yang berbeda. Input digital tidak bisa melakukan ini karena hanya membedakan HIGH dan LOW.

Tegangan 3,30 V di Serial Monitor tidak berarti tegangan pin tepat 3,3 V. Rumus `raw * (3.3/4095.0)` selalu mengubah 4095 menjadi 3,30 V, padahal ADC ESP32 sudah mentok di 4095 sebelum tegangan mencapai 3,3 V karena responsnya tidak linear di ujung atas rentang. ADC ESP32 juga kurang linear di dekat 0 V, sehingga tegangan 0,10–0,20 V pada kondisi gelap hanya perkiraan. Fungsi `analogReadMilliVolts()` memakai data kalibrasi chip dan memberi tegangan yang lebih akurat. Jika LDR perlu membedakan tingkat cahaya terang, resistor pembagi yang lebih kecil dari 10 k$\Omega$ bisa

dipakai agar ADC tidak cepat jenuh. GPIO 34 termasuk ADC1, jadi tetap bisa dipakai saat WiFi aktif, berbeda dengan pin ADC2.

# 3 Hasil Tugas Modul

## 3.1 Rancangan Sistem

Tugas modul meminta simulasi matrix pushbutton 3×3 yang mengendalikan matrix LED 3×3 pada ESP32, dengan debouncing di setiap tombol. Dalam rancangan ini, setiap tekanan tombol yang valid membalik status LED di posisi yang sama (menyala atau mati). Dengan logika toggle, bouncing langsung terlihat: tanpa debounce, satu tekanan bisa membalik LED beberapa kali dan status akhirnya acak. Simulasi dibuat di Wokwi / Cirkit / Tinkercad.

Matrix pushbutton terdiri dari 3 jalur baris dan 3 jalur kolom, dan setiap tombol menghubungkan satu baris dengan satu kolom. Pin kolom memakai `INPUT_PULLUP`. Program memindai baris satu per satu dengan memberi LOW pada baris aktif, sementara baris lain dibuat `INPUT` (high-Z) supaya dua baris tidak terhubung singkat jika dua tombol di kolom yang sama ditekan bersamaan. Jika kolom $c$ terbaca LOW saat baris $r$ aktif, berarti tombol $(r, c)$ ditekan.

Pada matrix LED, anoda di setiap baris disatukan, dan katoda di setiap kolom disatukan lalu dihubungkan ke ESP32 melalui resistor 220 Ω. LED dinyalakan dengan multiplexing: satu baris diberi HIGH, kolom yang LED-nya harus menyala diberi LOW, lalu baris berikutnya diaktifkan 2 ms kemudian. Pergantian ini cukup cepat sehingga semua LED terlihat menyala bersamaan.

**Tabel 6:** Alokasi pin ESP32 pada Tugas Modul

| Fungsi | GPIO (1, 2, 3) | Konfigurasi |
|---|---|---|
| Baris tombol | 13, 14, 27 | LOW saat dipindai, `INPUT` (high-Z) saat tidak dipindai |
| Kolom tombol | 26, 25, 33 | `INPUT_PULLUP`, HIGH saat tombol dilepas |
| Baris LED (anoda) | 23, 22, 21 | Output aktif HIGH, satu baris aktif bergantian |
| Kolom LED (katoda) | 19, 18, 5 | Output aktif LOW melalui resistor 220 Ω |

## 3.2 Source Code

**Listing 7:** Matrix pushbutton 3x3 mengendalikan matrix LED 3x3 dengan debouncing

```
#include <Arduino.h>

// ---------------- Alokasi pin ----------------
const uint8_t KEY_ROW[3] = {13, 14, 27}; // baris tombol (dipindai LOW)
const uint8_t KEY_COL[3] = {26, 25, 33}; // kolom tombol (INPUT_PULLUP)
const uint8_t LED_ROW[3] = {23, 22, 21}; // anoda LED, aktif HIGH
const uint8_t LED_COL[3] = {19, 18, 5};  // katoda LED (via 220 ohm), aktif LOW

#define DEBOUNCE_DELAY    50 // ms, sama seperti Percobaan 4
#define LED_SCAN_INTERVAL 2  // ms per baris LED (multiplexing)

// ---------------- Status ----------------
bool ledState[3][3] = {{false}};          // status setiap LED
int lastFlickerableState[3][3];           // pembacaan mentah sebelumnya
int stableState[3][3];                    // pembacaan yang sudah stabil
unsigned long lastDebounceTime[3][3];     // waktu perubahan terakhir
```

```
17
18 uint8_t activeLedRow = 0;
19 unsigned long lastLedScan = 0;
20
21 void setup() {
22   Serial.begin(115200);
23   for (int i = 0; i < 3; i++) {
24     pinMode(KEY_ROW[i], INPUT);           // high-Z saat tidak dipindai
25     pinMode(KEY_COL[i], INPUT_PULLUP);    // kolom ditarik ke HIGH
26     pinMode(LED_ROW[i], OUTPUT);
27     digitalWrite(LED_ROW[i], LOW);        // semua baris LED mati
28     pinMode(LED_COL[i], OUTPUT);
29     digitalWrite(LED_COL[i], HIGH);       // semua kolom LED mati
30   }
31   for (int r = 0; r < 3; r++) {
32     for (int c = 0; c < 3; c++) {
33       lastFlickerableState[r][c] = HIGH;
34       stableState[r][c] = HIGH;
35       lastDebounceTime[r][c] = 0;
36     }
37   }
38   Serial.println("Matrix 3x3 siap. Tekan tombol untuk toggle LED.");
39 }
40
41 // Memindai matrix tombol dan menerapkan debouncing pada setiap tombol
42 void scanKeys() {
43   for (int r = 0; r < 3; r++) {
44     pinMode(KEY_ROW[r], OUTPUT);          // aktifkan baris r
45     digitalWrite(KEY_ROW[r], LOW);
46     delayMicroseconds(10);                // tunggu sinyal kolom stabil
47
48     for (int c = 0; c < 3; c++) {
49       int reading = digitalRead(KEY_COL[c]);
50
51       if (reading != lastFlickerableState[r][c]) {
52         lastDebounceTime[r][c] = millis(); // sinyal berubah: reset pewaktu
53       }
54       if ((millis() - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
55         if (reading != stableState[r][c]) {
56           stableState[r][c] = reading;
57           if (stableState[r][c] == LOW) {  // tekanan valid
58             ledState[r][c] = !ledState[r][c];
59             Serial.printf("Tombol (%d,%d) -> LED (%d,%d) %s\n",
60                           r + 1, c + 1, r + 1, c + 1,
61                           ledState[r][c] ? "NYALA" : "MATI");
62           }
63         }
64       }
65       lastFlickerableState[r][c] = reading;
66     }
67     pinMode(KEY_ROW[r], INPUT);           // kembalikan baris ke high-Z
68   }
69 }
70
```

```
// Multiplexing matrix LED: satu baris aktif bergantian setiap 2 ms
void refreshLeds() {
  if (millis() - lastLedScan < LED_SCAN_INTERVAL) return;
  lastLedScan = millis();

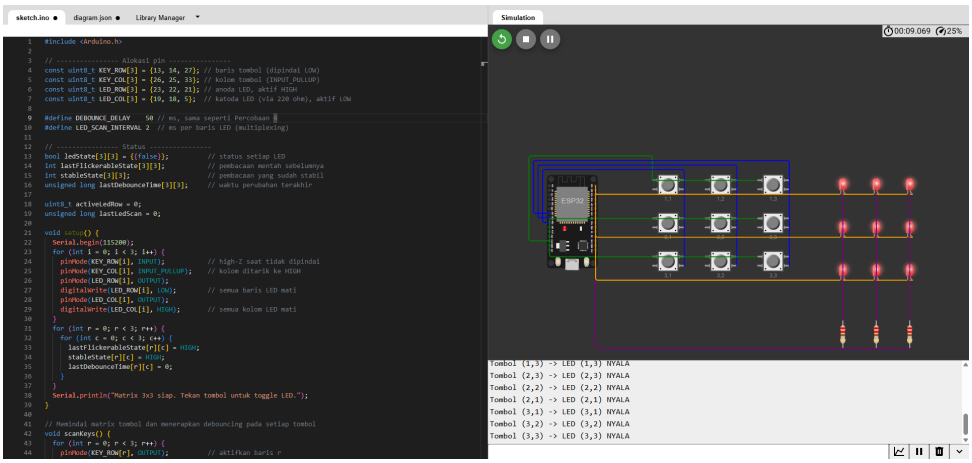
  digitalWrite(LED_ROW[activeLedRow], LOW);      // matikan baris sebelumnya
  activeLedRow = (activeLedRow + 1) % 3;
  for (int c = 0; c < 3; c++) {
    digitalWrite(LED_COL[c], ledState[activeLedRow][c] ? LOW : HIGH);
  }
  digitalWrite(LED_ROW[activeLedRow], HIGH);     // nyalakan baris berikutnya
}

void loop() {
  scanKeys();
  refreshLeds();
}
```

## 3.3 Penjelasan Program

Fungsi `scanKeys()` mengaktifkan baris tombol satu per satu dengan memberi LOW, membaca ketiga kolom, lalu mengembalikan baris itu ke `INPUT`. Setiap tombol punya data debounce sendiri dalam array 3×3 (`lastFlickerableState`, `stableState`, `lastDebounceTime`) dengan algoritma yang sama seperti Percobaan 4. Status LED baru dibalik ketika `stableState` berubah menjadi LOW setelah sinyal stabil lebih dari 50 ms. Fungsi `refreshLeds()` menjalankan multiplexing dengan `millis()`, bukan `delay()`, sehingga pemindaian tombol dan penyalaan LED bisa berjalan bergantian tanpa saling menunggu.

## 3.4 Hasil Simulasi




**Gambar 1:** Rangkaian simulasi matrix pushbutton 3×3 dan matrix LED 3×3 pada ESP32

## 3.5 Jawaban Pertanyaan Analisis

**1. Mengapa debouncing tetap diperlukan walaupun biasanya hanya satu tombol yang ditekan?** Bouncing berasal dari kontak mekanik di dalam setiap tombol, bukan dari jumlah tombol yang ditekan. Saat kontak logam bersentuhan atau terlepas, kontak itu bergetar dan menghasilkan beberapa transisi HIGH–LOW selama beberapa milidetik (Dasar Teori C.5). Satu tombol yang ditekan

sendirian tetap memantul, baik saat ditekan maupun saat dilepas. Pemindaian matrix sangat cepat, sehingga pembacaan bisa jatuh di tengah pantulan. Karena program memakai logika toggle, setiap transisi palsu membalik status LED, dan satu tekanan bisa membuat LED menyala lalu mati lagi. Debouncing membuat satu tekanan hanya mengubah status LED satu kali.

**2. Konfigurasi pull-up/pull-down pada kolom tombol, kondisi pin saat tombol dilepas, dan alasan tidak boleh floating.** Rancangan ini memakai pull-up internal (`INPUT_PULLUP`) di ketiga pin kolom (GPIO 26, 25, 33). Saat tombol dilepas, pin kolom terbaca HIGH karena ditarik ke 3,3 V. Saat tombol $(r, c)$ ditekan dan baris $r$ sedang diberi LOW, kolom $c$ ikut tertarik ke LOW. Tanpa pull-up, kolom yang tombolnya tidak ditekan tidak terhubung ke tegangan apa pun, karena baris yang tidak dipindai berada dalam kondisi high-Z. Pin floating mudah terpengaruh derau dan bisa terbaca HIGH atau LOW secara acak, sehingga program mendeteksi tekanan yang tidak pernah terjadi dan LED berubah sendiri. Debouncing tidak menolong di sini karena derau pada pin floating bisa bertahan lebih lama dari periode debounce.

**3. Perbedaan membaca GPIO sebagai digital input dan analog input (ADC), dan mengapa ADC bisa merepresentasikan lebih dari dua kondisi.** Input digital membandingkan tegangan pin dengan ambang tertentu: di atas ambang atas terbaca HIGH, di bawah ambang bawah terbaca LOW. Hasilnya hanya 1 bit, cocok untuk tombol yang memang hanya punya dua keadaan. Jika LDR dibaca sebagai input digital, sistem hanya tahu terang atau gelap, dan tegangan di antara kedua ambang memberi pembacaan yang tidak menentu. ADC mengukur besar tegangan dan menguantisasinya menjadi bilangan $n$-bit. ADC ESP32 12-bit membagi 0–3,3 V menjadi $2^{12} = 4096$ tingkat dengan resolusi sekitar 0,8 mV. Resistansi LDR berubah terus-menerus mengikuti cahaya, begitu juga tegangan pembagi tegangannya, sehingga ADC bisa menunjukkan seberapa terang cahaya yang diterima, bukan hanya terang atau gelap.

# 4 Kesimpulan

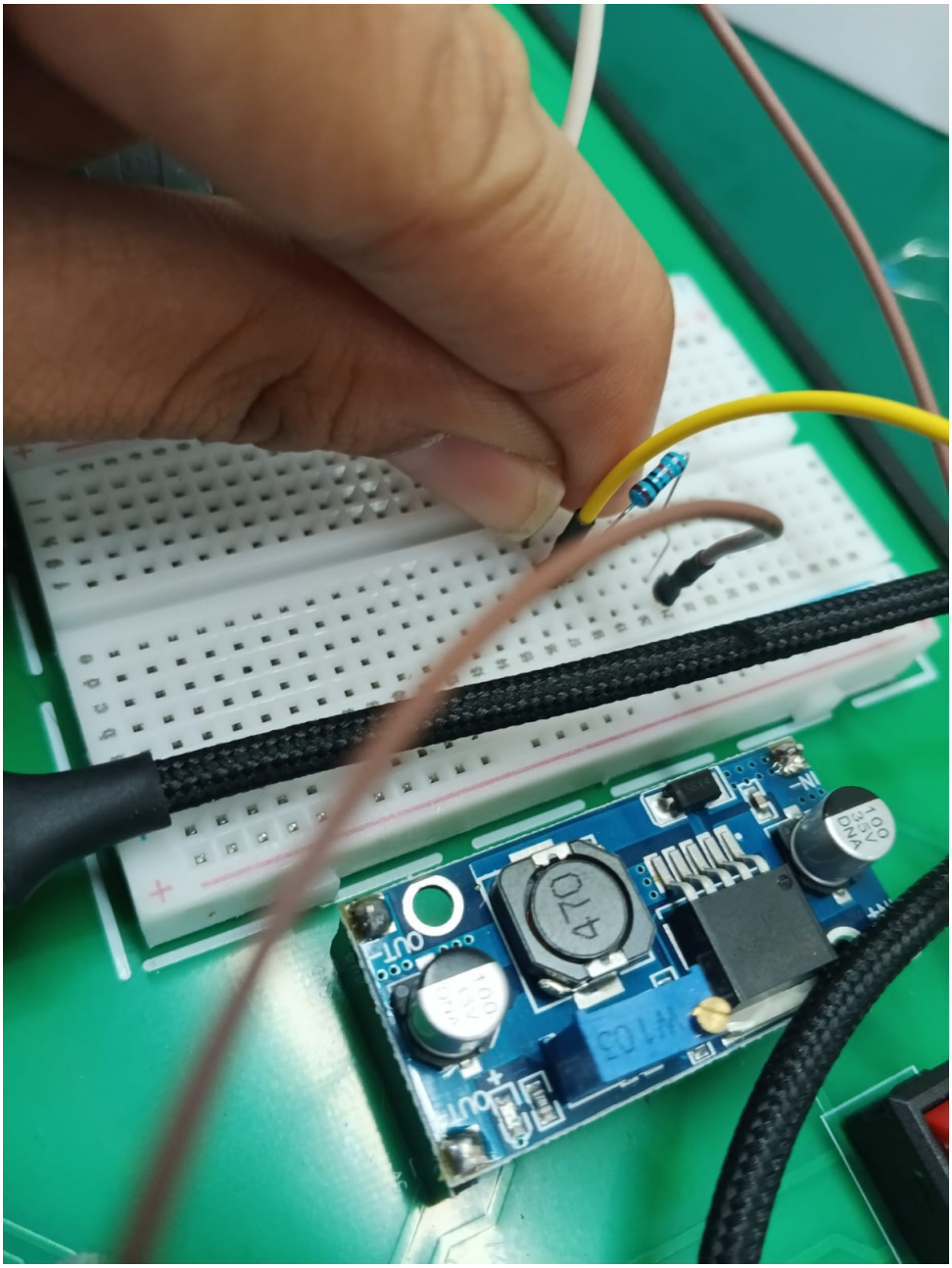
Berdasarkan hasil percobaan yang dilakukan, PlatformIO di VSCode terbukti dapat digunakan untuk memprogram mikrokontroler STM32 dan ESP32 menggunakan framework Arduino maupun ESP-IDF dalam satu proyek yang sama, cukup dengan menyesuaikan konfigurasi pada file `platformio.ini`. Pada pengujian dasar, program Blink berhasil diunggah ke STM32 Blackpill melalui ST-Link, di mana LED bawaan pada pin PC13 menyala saat diberi logika LOW karena bersifat aktif LOW. Sementara itu, program Blink dengan framework ESP-IDF juga berhasil dijalankan pada ESP32 dengan struktur yang berbeda dari Arduino, yakni menggunakan `app_main()` sebagai task FreeRTOS, konfigurasi GPIO yang lebih eksplisit, serta fungsi `vTaskDelay()` untuk pengaturan jeda.

Selanjutnya, pengujian input digital menunjukkan bahwa konfigurasi pull-up membuat pin terbaca HIGH saat tombol dilepas dan LOW saat ditekan, sedangkan pull-down bekerja sebaliknya. Karena konfigurasi eksternal dan internal memberikan hasil yang identik, fitur pull internal pada ESP32 dapat dimanfaatkan untuk menggantikan resistor eksternal pada GPIO 32 dan 33. Untuk mengatasi pembacaan ganda akibat pantulan mekanis kontak tombol, penerapan teknik debouncing berbasis `millis()` dengan jeda 50 ms berhasil memastikan setiap tekanan tombol hanya terhitung satu kali.

Pada bagian pembacaan analog dan integrasi sistem, nilai ADC dari sensor LDR tercatat berkisar antara 130–235 saat sensor ditutup, serta mencapai batas atasnya di kisaran 4089–4095 saat disinari

lampu. Hal ini memperlihatkan kemampuan input analog dalam membaca ribuan tingkat tegangan, berbeda dengan input digital yang hanya mengenali kondisi HIGH dan LOW. Terakhir, pada pengerjaan tugas modul, rangkaian matrix pushbutton 3×3 berhasil mengendalikan matrix LED 3×3 secara akurat melalui kombinasi teknik scanning, multiplexing, pemanfaatan pull-up internal, dan debouncing pada masing-masing tombol.

# 5 Lampiran




**Gambar 2:** Foto pada saat melakukan praktikum